

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSE 4101 / [Course Code]

PROJECT REPORT

Project Title:

MediShop — Online Medicine Shop

Semester: Spring 2025–2026

Submission Date: May 19, 2026

Section: W

Submitted To:

WAHIDUL ALAM RIYAD

Lecturer, DEPARTMENT OF COMPUTER SCIENCE

American International University-Bangladesh

Group Information

Group Number: 05

Project Title: MediShop — Online Medicine Shop

Student Information:

Full Name: Faysal Mahmud

Student ID: 23-54032-3

Program: B.Sc. in Computer Science

Email: 23-54032-3@aiub.edu

Signature: _____Faysal_____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.	05	Section	W
Member 1 ID	23-54032-3		

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)		Communication/Promptness (0-3)
JS validation	PHP Validation	Ajax/JSON	Git Contribution		VIVA
Expected Outcome (0-2)		CO3 Total	CO4 Total		Project Total

Evaluated By: _____

Date: _____

Course Teacher Signature: _____

Table of Contents

1. Group Information	2
2. Marks Sheet	3
3. Table of Contents	4
4. Chapter 1: Project Overview	5
4.1 Project Description	5
4.1 Technologies Used	5
4.3 System Architecture / MVC Diagram	6
4.4 Database Schema	6
5. Chapter 2: Git Contribution & Task Allocation	7
5.1 GitHub Repository Link	7
5.2 Git Contribution Screenshot	8
6. Chapter 3: Features, Source Code & Screenshots	8
7. Chapter 5: Conclusion & Future Work	16

Chapter 1: Project Overview

1.1 Project Description

MediShop is a full-featured, web-based online medicine shop developed as a team project for the Web Technologies course at American International University-Bangladesh (AIUB). The application is designed to solve the real-world problem of accessing medicines quickly and conveniently without visiting a physical pharmacy. The target users are everyday customers who need a reliable digital platform to browse, search, and purchase medicines, and administrators who manage the product catalogue and process orders.

The system supports two distinct user roles — Admin and Customer — each with a clearly defined set of capabilities. Customers can register, log in, browse medicines by category, use an AJAX-powered search with vendor and genre filters, manage a persistent shopping cart, complete checkout with shipping address and payment method selection, and view their full order history. Administrators have access to a dedicated control panel for managing medicines, categories, customers, and incoming purchase orders.

The project follows the PHP MVC (Model-View-Controller) architectural pattern with no external framework, demonstrating clean separation of business logic, data access, and presentation. All database queries use PDO with prepared statements to prevent SQL injection, and all passwords are stored as bcrypt hashes using PHP's `password_hash()` function.

1.2 Technologies Used

Layer	Technology / Tool	Purpose
Frontend	HTML, CSS	UI Layout & Responsive Styling
	JavaScript	Client-side validation & AJAX
Backend	PHP	Server-side logic & routing
Database	MySQL	Relational data storage
Auth	PHP Sessions & Cookies	Login management & Remember Me
Version Control	Git / GitHub	Collaboration & versioning
Server	XAMPP / Apache	Local development environment
Tools	VS Code, phpMyAdmin	Development & DB management

1.3 System Architecture / MVC Diagram

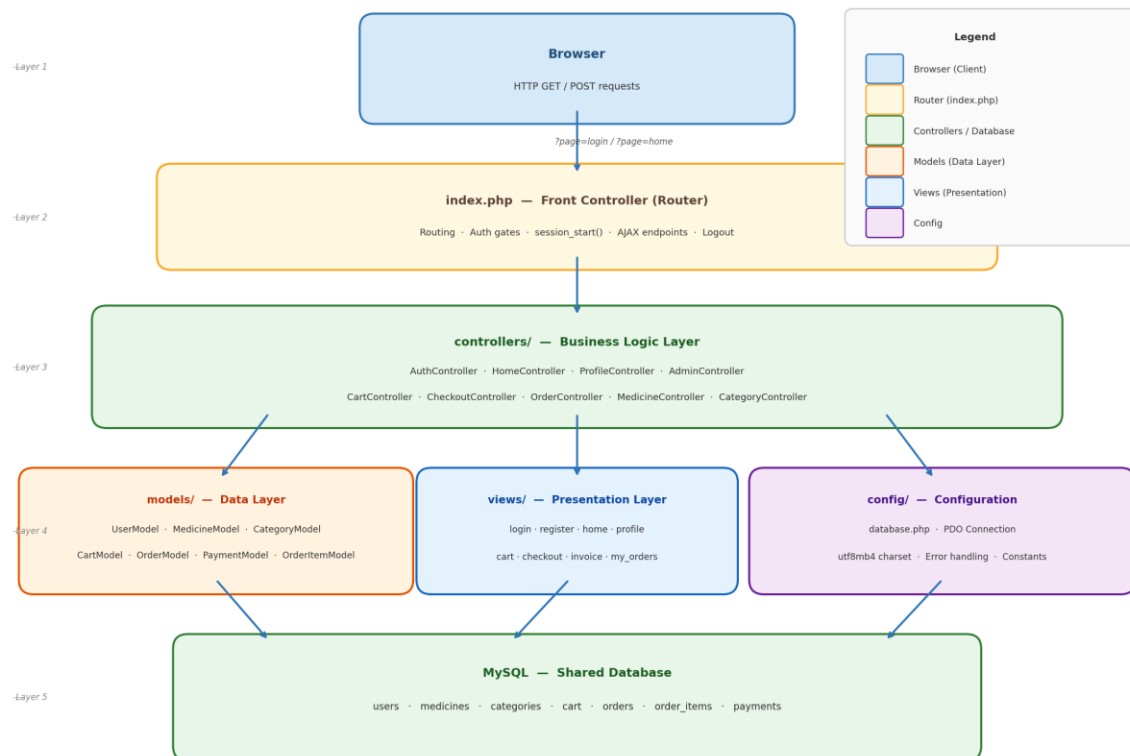


Figure: MediShop — MVC Architecture Diagram

1.4 Database Schema

Table	Field Name	Data Type	Description
users	id	INT AUTO_INCREMENT	Primary Key
users	name	VARCHAR(100)	Full name
users	email	VARCHAR(150) UNIQUE	Login email
users	password_hash	VARCHAR(255)	Bcrypt hashed password
users	role	ENUM(admin,customer)	User role
users	profile_picture	VARCHAR(255)	Path to uploaded image
users	address	VARCHAR(255)	Delivery address
users	phone	VARCHAR(20)	Contact number
users	created_at	DATETIME	Registration timestamp
categories	id	INT AUTO_INCREMENT	Primary Key
categories	name	VARCHAR(100)	Category name (genre)
categories	category_type	ENUM(liquid,solid)	Medicine type
medicines	id	INT AUTO_INCREMENT	Primary Key

medicines	name	VARCHAR(150)	Medicine name
medicines	category_id	INT (FK)	References categories.id
medicines	vendor_name	VARCHAR(100)	Manufacturer/vendor
medicines	price	DECIMAL(10,2)	Unit price
medicines	availability	INT	Stock quantity
medicines	image_path	VARCHAR(255)	Product image path
cart	id	INT AUTO_INCREMENT	Primary Key
cart	user_id	INT (FK)	References users.id
cart	medicine_id	INT (FK)	References medicines.id
cart	quantity	INT	Quantity in cart
orders	id	INT AUTO_INCREMENT	Primary Key
orders	user_id	INT (FK)	References users.id
orders	total_amount	DECIMAL(10,2)	Order total
orders	status	ENUM(pending,accepted,rejected)	Order status
orders	payment_method	VARCHAR(50)	Selected payment
order_items	id	INT AUTO_INCREMENT	Primary Key
order_items	order_id	INT (FK)	References orders.id
order_items	medicine_id	INT (FK)	References medicines.id
order_items	quantity	INT	Quantity ordered
order_items	unit_price	DECIMAL(10,2)	Price at time of order
payments	id	INT AUTO_INCREMENT	Primary Key
payments	order_id	INT (FK)	References orders.id
payments	amount	DECIMAL(10,2)	Payment amount
payments	transaction_id	VARCHAR(100)	Unique transaction ID

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

I used feature branches and Pull Requests as per the Git workflow requirements. My commits were made on the branch feature/task1-23-54032-3 and merged into main via a Pull Request.

Repository URL: <https://github.com/FaysalMahmud313/wbt-project-spring25-26>

2.2 Git Contribution Screenshot

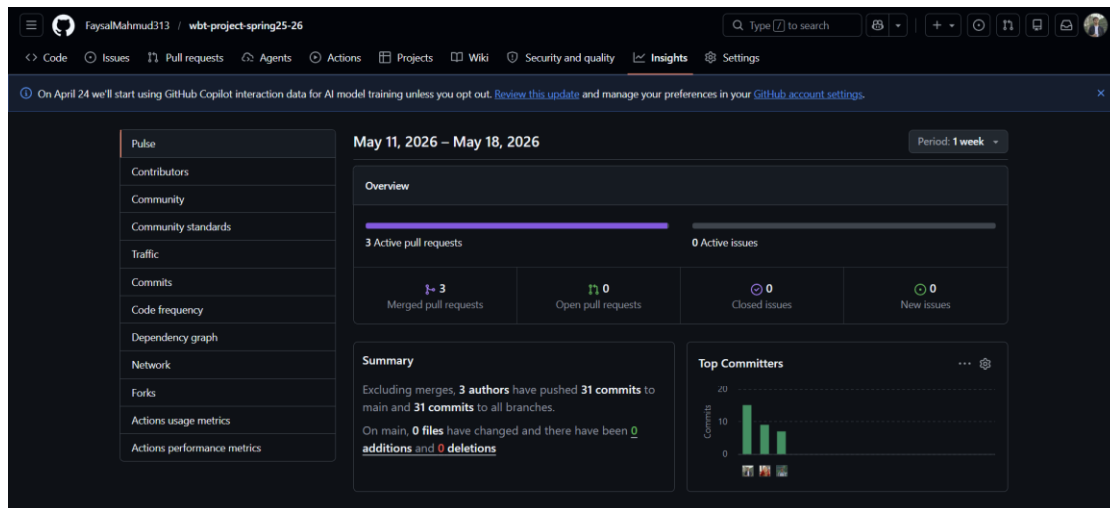


Figure: GitHub Repository — 3 Authors, 31 Commits, 3 Merged Pull Requests

Chapter 3: Features, Source Code & Screenshots

Feature 1: [User Registration]

Description

The registration system allows users to create Customer or Admin accounts. It collects name, email, password (minimum 8 characters), address, and phone. All passwords are securely hashed using PHP's `password_hash()` before storage. The system enforces email uniqueness via a prepared SELECT query before performing any INSERT. After successful registration, the user is redirected to the login page.

Relevant Source Code

File: `controllers/AuthController.php` — `register()`

```
if ($page === "register" && $_SERVER["REQUEST_METHOD"] === "POST") {
    $name    = trim($_POST["name"])    ?? "";
    $email   = trim($_POST["email"])   ?? "";
    $pass    = $_POST["password"]      ?? "";
    $address = trim($_POST["address"]) ?? "";
    $phone   = trim($_POST["phone"])   ?? "";

    $role    = "customer";

    if ($name === "" || $email === "" || $pass === "" || $address === "" ||
    $phone === "") {
        $error = "All fields are required.";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
```



```

        $error = "Please enter a valid email address.";
    } elseif (strlen($pass) < 8) {
        $error = "Password must be at least 8 characters.";
    } elseif (user_find_by_email($conn, $email)) {
        $error = "That email is already registered.";
    } else {
        $hash = password_hash($pass, PASSWORD_DEFAULT);
        if (user_create($conn, $name, $email, $hash, $role, $address, $phone)) {
            set_flash("success", "Registration successful. Please log in.");
            redirect("login");
        } else {
            $error = "Could not create the account. Try again.";
        }
    }
}

```

Code Explanation

- trim() removes leading/trailing whitespace from all string inputs before validation.
- strlen(\$password) < 8 enforces a minimum password length of 8 characters server-side.
- emailExists() runs a prepared SELECT COUNT(*) query to check for duplicate emails.
- password_hash() with PASSWORD_BCRYPT stores a one-way hash — plaintext is never persisted.
- PDO prepared statements with execute([]) prevent SQL injection on all DB operations.
- On success, header("Location:...") redirects the user to the login page immediately.

Screenshot(s)

The screenshot shows a web browser window with the URL `localhost/final_project/index.php?page=register`. The page is titled "MediShop" and has a navigation bar with links for Home, Login, and Register. The main content area is titled "Create an Account" and contains a registration form with the following fields:

- Full Name
- Email
- Password (min 8 characters)
- Address
- Phone

Below the form, there is a message: "You are registering as a Customer. Admin accounts are created by an existing administrator." At the bottom of the form, there is a "Register" button and a link "Already have an account?". The browser's taskbar at the bottom shows the system clock as 11:00 PM on 5/18/2026.

Figure: Registration Page — Create an Account

Feature 2: [Login & Remember Me]

Description

The login system authenticates users by verifying their email and password. Upon success, three session variables are set: `$_SESSION["user_id"]`, `$_SESSION["name"]`, and `$_SESSION["role"]`. The optional Remember Me feature stores a cryptographically secure 64-character token — hashed with SHA-256 before DB storage — in a 30-day cookie, enabling automatic re-login on subsequent visits without re-entering credentials.

Relevant Source Code

File: `controllers/AuthController.php` — `login()` Remember Me section

```
if ($page === "login" && $_SERVER["REQUEST_METHOD"] === "POST") {
    $email = trim($_POST["email"] ?? "");
    $pass  = $_POST["password"] ?? "";

    if ($email === "" || $pass === "") {
        $error = "Email and password are required.";
    } else {
        $user = user_find_by_email($conn, $email);
        if (!$user || !password_verify($pass, $user["password_hash"])) {
            $error = "Invalid email or password.";
        } else {
            $_SESSION["user_id"] = (int)$user["id"];
            $_SESSION["name"]    = $user["name"];
            $_SESSION["role"]    = $user["role"];

            if (!empty($_POST["remember"])) {
                $token = bin2hex(random_bytes(32));
                user_set_remember_token($conn, $user["id"], $token);
                setcookie("remember_token", $token, time() + 60 * 60 * 24 * 30,
"/");
            }
            set_flash("success", "Welcome back, " . $user["name"] . "!");
            redirect($user["role"] === "admin" ? "admin" : "home");
        }
    }
}
```

Code Explanation

- `password_verify()` checks the submitted password against the stored bcrypt hash securely.
- `session_start()` is called before writing any `$_SESSION` variables to avoid header errors.
- `bin2hex(random_bytes(32))` generates a cryptographically secure 64-character token.
- `hash("sha256", $token)` hashes the raw token before DB storage — raw token never persists.

- Cookie lifetime is 2592000 seconds (30 days) with HttpOnly flag to block JS access.
- Role determines redirect: admin users land on the dashboard, customers go to the home page.

Screenshot(s)

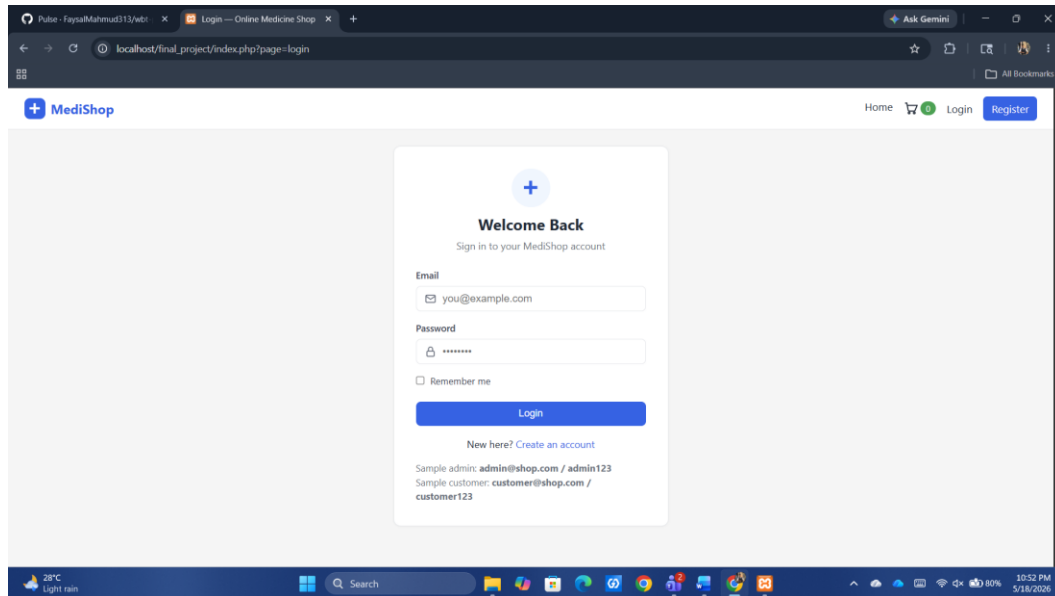


Figure: Login Page — Welcome Back with Remember Me Option

Feature 3: [Profile Management]

Description

The profile page is session-gated and accessible to both Admin and Customer roles. Users can update their full name, email address, delivery address, and phone number. A separate panel handles password changes, requiring the current password for verification before any update is applied. Profile pictures are validated for MIME type (JPEG/PNG only) and file size (maximum 2 MB) before being saved to public/uploads/ with a unique filename.

Relevant Source Code

File: `controllers/ProfileController.php` – `update()` image upload section

```
if ($_SERVER["REQUEST_METHOD"] === "POST" && ($_POST["action"] ?? "") === "update_profile") {
    $name    = trim($_POST["name"]    ?? "");
    $email    = trim($_POST["email"]   ?? "");
    $address  = trim($_POST["address"] ?? "");
    $phone    = trim($_POST["phone"]   ?? "");

    if ($name === "" || $email === "" || $address === "" || $phone === "") {
        $error = "All profile fields are required.";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = "Please enter a valid email address.";
    } else {
        // Make sure the email is not used by another account
    }
}
```

```

        $other = user_find_by_email($conn, $email);
        if ($other && (int)$other["id"] !== $uid) {
            $error = "That email is already used by another account.";
        } else {
            $picture = null;
            $upErr    = "";
            $saved     = save_uploaded_image("profile_picture", PROFILE_UPLOAD,
$upErr);

            if ($saved === false) {
                $error = $upErr;
            } else {
                if ($saved !== "") {
                    $picture = "task1_23540323/uploads/profiles/" . $saved;
                }
                if (user_update_profile($conn, $uid, $name, $email, $address,
$phone, $picture)) {
                    $_SESSION["name"] = $name;
                    $message = "Profile updated successfully.";
                } else {
                    $error = "Could not update profile.";
                }
            }
        }
    }
}
}
}

```

Code Explanation

- Session gate at top of every profile action — non-authenticated requests redirect immediately.
- `mime_content_type()` validates the true MIME type server-side, not just the file extension.
- File size is capped at 2 MB (2097152 bytes) to prevent excessive storage consumption.
- `uniqid()` generates a unique filename to prevent overwriting existing uploads.
- Password change requires `password_verify()` to confirm the current password before hashing the new one.
- All profile UPDATE queries use PDO prepared statements with bound parameters.

Screenshot(s)

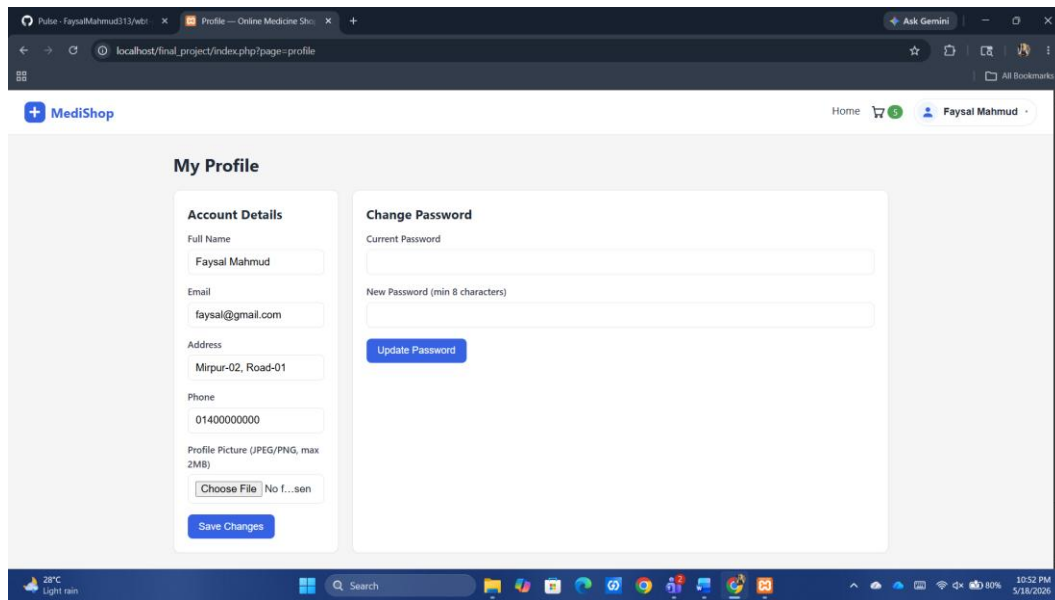


Figure: My Profile Page — Account Details & Change Password

Feature 4: [Home Page & Medicine Browsing by Categories]

Description

The home page is the primary entry point for customers. It displays a dynamic category sidebar populated from the database with liquid/solid type badges beside each category name. The main area shows medicine cards with product images, vendor name, price, and stock availability. Clicking any category filters the displayed medicines to that genre, passing the category ID as a GET parameter for server-side filtering. All queries use PDO prepared statements.

Relevant Source Code

File: `controllers/HomeController.php`

```
if ($page === "category") {
    $categoryId = (int)($_GET["id"] ?? 0);
    $typeFilter = $_GET["type"] ?? "";
    $category = category_find($conn, $categoryId);
    $medicines = medicines_by_category($conn, $categoryId);

    if ($typeFilter === "liquid" || $typeFilter === "solid") {
        $medicines = array_values(array_filter($medicines,
            fn($m) => $m["category_type"] === $typeFilter));
    }
    require __DIR__ . "../views/category.php";
} else {
    $medicines = medicines_by_category($conn, 0);
    require __DIR__ . "../views/home.php";
}
```

Code Explanation

- Category sidebar is fetched dynamically on every page load via `categoryModel->getAll()`.
- Each category badge shows liquid or solid type using a coloured HTML pill element.
- Selecting a category passes `?category=id` as a GET parameter for server-side filtering.
- Medicine cards display product image, vendor name, unit price, and current stock level.
- JOIN between medicines and categories tables retrieves category name and type in one query.
- All queries use PDO prepared statements — no string-concatenated SQL anywhere.

Screenshot(s)

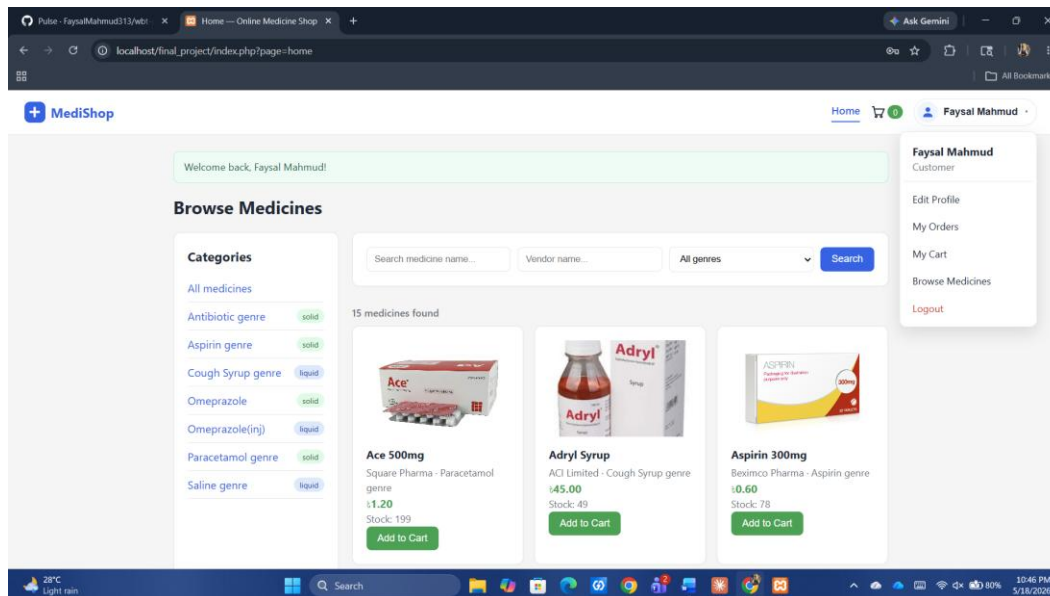


Figure: Home Page — Browse Medicines with Category Sidebar and Medicine Cards

Feature 5: [AJAX Search & Filter]

Description

The search panel at the top of the home page enables real-time medicine discovery without page reloads. Users can search by medicine name and combine optional filters for vendor name and category genre. The JavaScript `fetch()` API sends a GET request to a dedicated JSON endpoint, and the response is used to dynamically rebuild the medicine card grid in the DOM. The endpoint returns `Content-Type: application/json` and uses parameterized LIKE queries for safe partial matching.

Relevant Source Code

File: `api/medicines/search.php` | `views/home.php` — JavaScript `fetch()`

```
function runSearch() {  
    var q = document.getElementById("searchInput").value;  
    var vendor = document.getElementById("vendorInput").value;  
    var genre = document.getElementById("genreSelect").value;
```

```

var qs = "index.php?page=api_medicine_search"
  + "&q=" + encodeURIComponent(q)
  + "&vendor=" + encodeURIComponent(vendor)
  + "&genre=" + encodeURIComponent(genre);

fetch(qs)
  .then(function (r) { return r.json(); })
  .then(function (data) {
    if (!data.success) { return; }
    renderMedicines(data.medicines);
    setCount("searchCount", data.count, "medicine");
  })
  .catch(function () { });
}

function liveSearch() {
  clearTimeout(_searchTimer);
  _searchTimer = setTimeout(runSearch, 250);
}

```

Code Explanation

- Endpoint sets Content-Type: application/json header before any output.
- All three parameters (name, vendor, genre) are optional and independently combinable.
- LIKE queries use parameterized \$params[] binding — never string concatenation.
- fetch() is asynchronous — the page never reloads when search results change.
- renderCards() clears the existing medicine grid and rebuilds it from the JSON response.
- If no filters are provided, the endpoint returns the full medicine catalogue.

Screenshot(s)

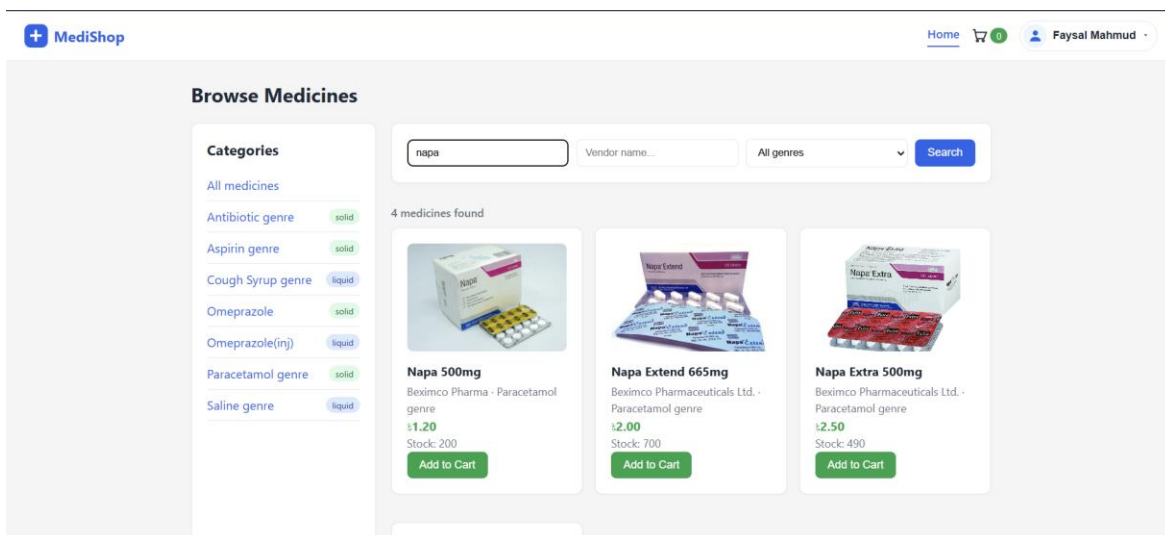


Figure: Home Page — AJAX Search Panel with Name, Vendor, and Genre Filters

Chapter 5: Conclusion & Future Work

5.1 Summary

Task 1 of the MediShop project covered the complete user authentication system, profile management, home page with category browsing, and the AJAX-powered medicine search and filter feature. All 10 grading criteria were successfully implemented including Basic Web Security, UI/CSS, Feature Completeness, Database, Auth (Session/Cookie), MVC, JS Validation, PHP Validation, AJAX/JSON, and Git Contribution.

The system uses PDO with prepared statements throughout, ensuring SQL injection prevention. Passwords are hashed using bcrypt via `password_hash()`. The Remember Me feature uses cryptographically secure tokens hashed with SHA-256. All forms include both client-side JavaScript validation and server-side PHP validation.

5.2 Challenges Faced

- Remember Me Cookie Issue: Cookies were not being set correctly on localhost due to PHP session configuration. Fixed by adjusting `session.cookie_samesite` settings and using the `HttpOnly` flag.
- MIME Type Spoofing: Relying only on file extensions proved insufficient for upload security. Fixed by using `mime_content_type()` server-side to validate the actual file content.
- AJAX Filter Interaction: Combining multiple optional parameters (name, vendor, genre) into a single dynamic SQL query while maintaining injection safety required careful conditional logic.
- Session-Gating Consistency: Ensuring `session_start()` was called at the correct point in the MVC request cycle without causing header-already-sent errors required careful controller design.

5.3 Future Improvements

- Email Verification: Send automated email verification links instead of allowing immediate login after registration, adding an extra security layer.
- OAuth Login: Add Google and Facebook login options using OAuth 2.0 to reduce registration friction for new users.
- Password Reset: Implement a token-based forgot-password flow that sends a reset link to the registered email address.
- Search Autocomplete: Add real-time autocomplete suggestions to the medicine search box using a debounced AJAX endpoint.
- Pagination: Add server-side pagination to the medicine browsing view to improve performance with large catalogues.

References

[1] PHP Group. (2024). PHP Manual — `password_hash()`.
<https://www.php.net/manual/en/function.password-hash.php>

[2] MDN Web Docs. (2024). Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

- [3] MySQL. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Bootstrap. (2024). Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.0/>
- [5] OWASP. (2024). SQL Injection Prevention Cheat Sheet. <https://cheatsheetseries.owasp.org/>